Diffusion Models and Recurrent Neural Networks

Due at 6:00pm on Friday Nov 28

What you need to get

Accept the assignment on GitHub Classroom using this link. Click to accept the invitation, and then follow the directions. You will then be able to clone the assignment repository.

What to submit

You must submit your solutions electronically, as follows.

- For **"handwritten" questions**, submit your solutions to Crowdmark. You can typeset your solution (eg. using LaTeX or Word), or write your solution on paper or on a tablet. **If you submit photos of handwritten work, it is your responsibility to ensure that they are legible.**
- For each **coding question**, you must accept the invitation from **github classroom**, and follow the directions to clone the assignment's repository. Edit the `README.md` file to **add your name and student number**. You will submit your solutions by uploading a **PDF** version of your answers on Crowdmark and by pushing your completed code back to the repo. We suggest you push your code frequently to demonstrate your thought process and to avoid your assignment being flagged for an academic integrity check. You also need to include a link to your Github repository in your Crowdmark submission.

Note: The use of GenAI to code logic is not allowed in this course (see the Outline of this course). If you choose to use Colab, you will need to deactivate the AI assistance option in Tools > Settings > AI Assistance. The same comment holds for other AI assistance tools.

What to do

1. **[6 marks] Diffusion models** are powerful generative methods that successively transform an initial data sample x_0 into a sequence of increasingly noisy samples $\{x_1, x_2, \ldots, x_T\}$. While the *forward* process (adding noise at each step) is straightforward, sampling in reverse requires us to *denoise* the data step by step. Mathematically, this involves deriving the reverse-process distribution

$$q\big(x_{t-1} \mid x_t, x_0\big).$$

   (a) **[4 marks]** In the diffusion model lecture, we have looked at the forward distributions

$$q\big(x_t \mid x_{t-1}, x_0\big) \quad \text{and} \quad q\big(x_t \mid x_0\big).$$

   Show that $q(x_{t-1} \mid x_t, x_0)$ must be Gaussian in x_{t-1}. Note that you do not need to fully determine its formula.

   Hint: Use Bayes' theorem. You also can define the probability distribution $q_{x_0}\big(x_t \mid x_{t-1}\big) \equiv q\big(x_t \mid x_{t-1}, x_0\big)$.

   (b) **[2 marks]** If we assume the standard Gaussian form,

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}(x_t, x_0), \tilde{\beta}_t I),$$

   we can derive explicit expressions for the mean and variance,

$$\tilde{\mu}(x_t, x_0) = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_t\right), \tag{1}$$

$$\tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t,$$

where $\alpha_t = 1 - \beta_t$ are forward process parameters, $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s$, and ϵ_t is the noise added in the forward process at step t. There is no need to show this result. In practice, diffusion models learn an approximate reverse process distribution $p_\theta(x_{t-1} \mid x_t)$ to match the true reverse distribution $q(x_{t-1} \mid x_t, x_0)$. Let us assume that $p_\theta(x_{t-1} \mid x_t)$ is Gaussian,

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta, \Sigma_\theta),$$

where

$$\mu_\theta = \frac{1}{\sqrt{\alpha_t}}\left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\epsilon_\theta(x_t, t)\right). \tag{2}$$

If the variances of $q(x_{t-1} \mid x_t, x_0)$ and $p_\theta(x_{t-1} \mid x_t)$ are equal (i.e., $\tilde{\beta}_t I = \Sigma_\theta$), then using Equations (1), (2), show that:

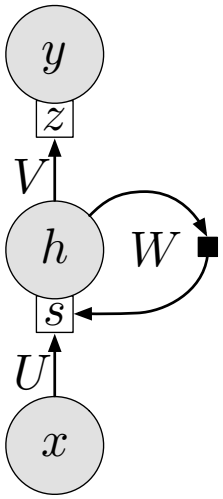
$$D_{\mathrm{KL}}(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t)) \propto \|\epsilon_t - \epsilon_\theta(x_t, t)\|_2^2,$$

where $\propto$ means "proportional to". Deriving the proportionality constant is not needed. You can use the following theorem:

Theorem 1 *The KL divergence between two Gaussians $q = \mathcal{N}(\mu_q, \Sigma_q)$ and $p = \mathcal{N}(\mu_p, \Sigma_p)$ with diagonal covariance matrices is given by:*

$$D_{\mathrm{KL}}(q\|p) = \frac{1}{2}\sum_{i=1}^{d}\left[\frac{\Sigma_{q,i}}{\Sigma_{p,i}} + \frac{(\mu_{q,i}-\mu_{p,i})^2}{\Sigma_{p,i}} - 1 + \log\frac{\Sigma_{p,i}}{\Sigma_{q,i}}\right].$$

2. **[7 marks] RNN Vanishing Gradient**. Check out the recurrent neural network in the figure on the right, where $x, h, y \in \mathbb{R}$. This network is governed by the equations



$$\begin{aligned} s_i &= x_i U + h_{i-1} W + b \\ h_i &= \sigma(s_i) \\ z_i &= h_i V + d \\ y_i &= \sigma(z_i) \end{aligned}$$

where σ is the logistic function, U, W, V, b, and d are scalars, and the superscript i refers to time step. Furthermore, suppose the objective function is

$$E(y_1, y_2, \ldots, t_1, t_2, \ldots) = \sum_{i=1} L(y_i, t_i)\,.$$

(a) **[3 marks]** Suppose you are given $\frac{\partial E}{\partial h_{i+1}}$. Prove that

$$\frac{\partial E}{\partial h_i} = \frac{\partial L(y_i, t_i)}{\partial y_i}\sigma'(z_i)\,V + \frac{\partial E}{\partial h_{i+1}}\sigma'(s_{i+1})\,W$$

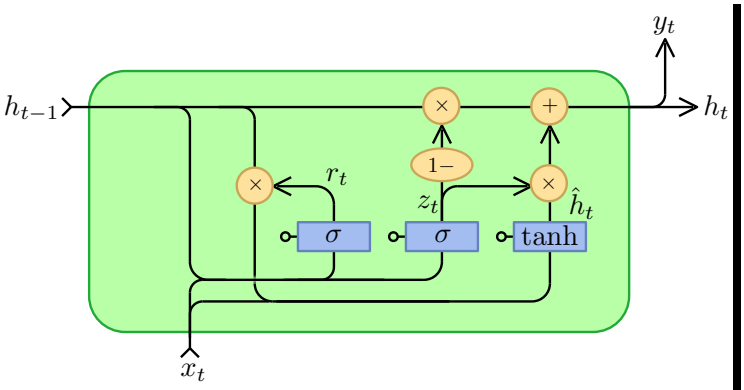
Show how you arrived at your answer.

(b) **[4 marks]** Suppose you unroll the network for T steps in order to do Backprop Through Time (BPTT). Assuming $|W| < 4$, show that as $T \to \infty$ the influence of $\frac{\partial L(y_T, t_T)}{\partial y_T}$ on $\frac{\partial E}{\partial h_1}$ goes to zero. As usual, show your work.

3. **[15 marks]** In this question, you will implement a Gated Recurrent Unit (GRU) network. The figure below shows a schematic of a GRU; the corresponding mathematical formulation is:

$$
\begin{aligned}
z_t &= \sigma\left(x_t U_z + h_{t-1} W_z + b_z\right) \\
r_t &= \sigma\left(x_t U_r + h_{t-1} W_r + b_r\right) \\
\hat{h}_t &= \tanh\left(x_t U_h + \left(r_t \odot h_{t-1}\right) W_h + b_h\right) \\
h_t &= \left(1 - z_t\right) \odot h_{t-1} + z_t \odot \hat{h}_t \\
y_t &= \text{Softmax}\left(h_t V + b_y\right)
\end{aligned}
$$

where σ is the logistic function, and ϕ is the logarithm of the softmax function, $U_\square$, $W_\square$, and V are connection-weight matrices, and $b_\square$ are biases.



Gated Recurrent Unit. Adapted from Jeblad under Creative Commons License

Much of the functionality for running and training a GRU (using backprop through time, or BPTT) is implemented in the class `GRU` in the notebook. However, parts of it still need to be filled in.

(a) **[11 marks]** Complete the implementation of the class by completing the following functions, as described by their docstrings:

(a) `__init__`: sets up all the weight matrices and activation functions

(b) `step`: takes one timestep and produces the output, y_t, from the hidden state

(c) `forward`: processes all the timesteps for the sequences in the batch, and returns their outputs

(d) `predict`: Given a sequence, it predicts the next n outputs. After taking in the input sequence, each step's output is used as input for the next step.

Note that you should **NOT** use PyTorch's `nn.GRU` or `nn.GRUCell` functions. You must implement the GRU yourself, using other, more basic PyTorch modules like `nn.Linear` and `nn.Sigmoid`, etc.

(b) **[1 mark]** Train your GRU network on the characters from the dataset taken from the book of 'Albert Einstein' *Relativity: The Special and the General Theory (1916)*. The necessary line of code is already included in the notebook. You might have to do some hyperparameter tuning.

(c) **[2 marks]** Add some lines to the notebook to evaluate how often the GRU network predicts the correct next letter after being fed in a sequence. Do this by feeding every sequence into the network and checking to see if the final output matches the target output. If the network has learned well, your fraction should be over 95%.

(d) **[1 mark]** Use your code to generate predicted text from a prompt. You can either choose one of the training sequences as a prompt, or you can enter your own prompt (please be respectful). You can choose 250 as the length of your generated text.